

Prefill once, reuse anywhere: non-prefix KV reuse for agent memory in vLLM

CS 6220 Course Project Proposal, Fall 2026, Group 7

Atharva Mohite, Teppei Kawashima, Prakhar Langer, Vishal Puneyani

1. Motivation and objectives

Before an LLM produces its first token it runs *prefill*, one pass over the prompt that computes a key and a value vector for every token at every layer (the KV cache). Prefill cost grows with prompt length and dominates time to first token (TTFT) for long prompts.

Memory-augmented agents produce exactly these prompts. For each question the agent retrieves relevant entries from its long-term memory and places them ahead of the question. Over a long conversation the same entries come back many times, in different combinations and orders, and the serving engine prefills them from scratch each time. vLLM [1] reuses KV only through *prefix caching*, which needs two prompts to start with identical tokens. If one question retrieves memories A, B, C and the next retrieves C, A, D, the prompts diverge right after the system prompt, and A and C are prefilled again.

Reusing a chunk’s KV at a new position is not exact. A token’s KV depends on its position, which rotary position embeddings (RoPE) [2] let us correct exactly by re-rotating the cached keys. It also depends on the tokens before it, which no rotation fixes: a chunk cached on its own never attended to the memories now placed in front of it. That second error is usually small, because tokens attend mostly within their own chunk and the question is always computed fresh. EPIC [3], CacheBlend [4] and AgentKVShift [5] close the gap by recomputing a small subset of tokens, but each assumes the error lives in a different place, and they have not been compared on one model and one agent-memory workload. We also found no working open-source implementation for current vLLM: LMCache’s [6] blending example is reported out of date for recent vLLM (issue #4476), and its newer blend mode has no vLLM-side client in multi-process mode (issue #5101).

Research question. *How much prefill can non-prefix KV reuse remove from a memory-augmented agent, and how do different ways of correcting reused KV trade answer quality against latency and throughput?*

Objectives. We will deliver (1) a Transformers implementation of chunk-level KV reuse with RoPE re-rotation and four correction methods behind one shared recompute loop; (2) a controlled comparison of the four methods on LoCoMo [7], measuring F1 against recompute budget and prefill cost against the number of retrieved memories; and (3) a vLLM connector plugin that serves reused memory KV to an unmodified vLLM, with latency and throughput measured under a workload whose repetition rate we control. Together these tell someone serving agents or RAG which correction is worth its cost, and how often memories must repeat before non-prefix reuse beats the prefix cache vLLM already has.

2. Related work

Prefix caching. vLLM’s PagedAttention [1] shares fixed-size KV blocks across requests with identical prefixes, and SGLang’s RadixAttention [8] keeps cached prefixes in a radix tree. Both are exact, and both miss when reused text follows different preceding text, which is the normal case for retrieved memories.

Position-independent reuse. Prompt Cache [9] precomputes attention states for prompt modules at positions declared in a schema, without correcting for the cross-module attention it skips. CacheGen [10] compresses and streams KV so it can be loaded instead of recomputed, which is complementary to our work.

Correcting reused KV. EPIC [3] calls the setting position-independent caching. It observes that a chunk cached alone treats its first tokens as an attention sink [11], which is wrong once the chunk sits mid-prompt, so it recomputes

the first few tokens (at most 32) of every chunk after the first. CacheBlend [4] finds that about 10-15% of tokens deviate strongly from their fresh KV, picks them at an early layer and recomputes only those. On RAG benchmarks it reports 2.2-3.3 $\times$ lower TTFT than full prefill with quality within 0.02 F1. AgentKVShift [5] targets agent memory. Its spectral analysis finds that most of a chunk’s reuse error is one shared offset, so it recomputes a few probe tokens, estimates the per-layer mean shift and adds it to every other token. On LoCoMo with Mistral-7B-Instruct-v0.3 and the LiCoMemory memory system it reports 0.491 F1 against 0.509 for full recompute at a 10% budget (a 3.5% relative gap), and it reports 2-3.5 $\times$ prefill speedups on a single A100. It is our main comparison point.

Serving and agents. LMCache [6], from the same group as CacheBlend, is the reference implementation of vLLM’s KV-connector interface and the open-source vLLM integration of non-prefix reuse we build against. LoCoMo [7] provides very long multi-session conversations with QA annotations. MemGPT [12] and A-Mem [13] are memory systems that produce the retrieve-then-prompt pattern we target. KVCOMM [14] and KVCMA [15] share KV across agents and matter only for our stretch goal.

3. Proposed work

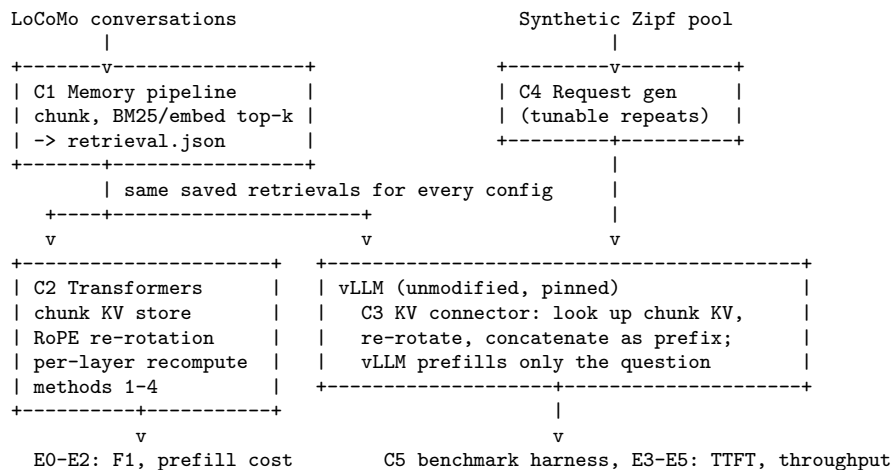
3.1 Approach

Each memory entry is a *chunk*. The first time a chunk is retrieved we prefill it alone and store its KV under a hash of its tokens. Later prompts containing it load that KV, re-rotate the keys to the new position and apply one of four corrections, which differ only in which tokens they recompute:

#	Method	Recomputes	Assumes the error is
1	Re-positioning only (baseline)	nothing	negligible
2	Boundary (EPIC)	first 16-32 tokens of each chunk	at chunk starts
3	Dynamic selection (CacheBlend)	10-15% of tokens deviating most at an early layer	sparse, anywhere
4	Offset correction (AgentKVShift)	a few probes; their mean shift is added to the rest	one shared shift per chunk

Every prompt is the system prompt, then the top- k retrieved entries, then the question, which is always computed fresh.

3.2 Architecture



C1, memory pipeline. Splits each LoCoMo conversation into entries of a few hundred tokens, retrieves the top- k per question with BM25 or embeddings, and saves the results to disk. Every configuration reads the same file, so no difference between methods can come from retrieval.

C2, Transformers implementation (the core). A chunk KV store keyed by token hash, RoPE re-rotation, and a custom forward pass that steps through the decoder layers and, at each layer, recomputes a token set S against the assembled cache and writes the fresh KV back. Each method is a function that returns S (plus, for method 4, an offset for the other tokens).

C3, vLLM connector plugin. vLLM can load an external KV connector that supplies cached KV for the first N tokens of a prompt. Ours builds that prefix from the re-rotated KV of the retrieved chunks, so vLLM computes only the question. This serves method 1 without touching vLLM’s source. Methods 2 to 4 recompute tokens inside the prefix at every layer, which would need scheduler changes, so they stay in Transformers.

C4, synthetic workload issues requests over a pool of entries with Zipf-distributed popularity; the exponent sets how often entries repeat. **C5, benchmark harness** drives vLLM’s benchmark tool with a custom dataset and records TTFT, latency and throughput.

3.3 Novelty and scope

The correction algorithms come from prior work, and we do not propose a new one. What we add: a like-for-like comparison (published results use different models, datasets and memory formats, while we run all four methods with one model, one retrieval output and one budget accounting); a new setting for EPIC and CacheBlend, which were evaluated on RAG passages rather than conversational memory; non-prefix reuse served through vLLM’s public plugin interface instead of engine changes; and a break-even analysis of how often memories must repeat before reuse beats prefix caching, which the papers above do not report.

The core is C1, C2 and experiments E0-E2, which answer the research question on their own. C3-C5 and E3-E5 start only after the E0 correctness gate passes. A multi-agent workload, where several agents read overlapping memories through one connector, is a stretch goal. Changing the source of vLLM, Transformers or the model is out of scope.

4. Plan of action

4.1 Resources

- **Hardware:** one A100-80GB or H100 per run on PACE ICE, through the course allocation.
- **Model and data:** Mistral-7B-Instruct-v0.3 [16] from HuggingFace, the model AgentKVShift reports on LoCoMo; LoCoMo from its public release (10 conversations, 1,986 questions).
- **Software:** PyTorch, Transformers, `rank_bm25`, sentence-transformers, and one pinned vLLM version recorded in a lock file. Code, configs and result logs live in a team GitHub repository.

The model’s bf16 weights need about 15 GB of GPU memory. Its KV takes 128 KB per token ($32 \text{ layers} \times 8 \text{ KV heads} \times 128 \text{ dims} \times K \text{ and } V \times 2 \text{ bytes}$), and LoCoMo conversations average about 16K tokens [7], so the KV for all ten conversations is about 20 GB and the chunk store fits in GPU or host memory without distributed storage.

4.2 Weekly schedule

Each component has an owner, and everyone reviews the E0 gate and the final numbers.

Week	Work	Milestone
6 (Sep 28-Oct 2)	Finalize scope, submit proposal	Proposal in
7 (Oct 5-9)	PACE setup, model and data download, full-prefill F1 harness (all)	Baseline F1
8 (Oct 12-16)	C1 and saved retrievals (Prakhar); KV store, re-rotation (Atharva); recompute loop (Teppe, Vishal)	Retrievals frozen
9 (Oct 19-23)	E0 tests; methods 1 and 2 (Atharva, Tepei)	E0 passes (gate)
10 (Oct 26-30)	Methods 3 and 4 (Teppe, Vishal); first E1 runs (Prakhar); study connector API (Atharva)	All methods run

Week	Work	Milestone
11 (Nov 2-6)	Full E1 sweep and E2 (Prakhar, Teppei, Vishal); connector prototype (Atharva)	Core E0-E2 done
12 (Nov 9-13)	Finish C3 (Atharva); C4 and C5 (Prakhar); E5 (Teppei); report outline (Vishal)	E5 passes
13 (Nov 16-20)	E3 and E4 (Atharva, Prakhar); ablations (Teppei, Vishal); draft report sections	All data in
14 (Nov 23-27)	Thanksgiving week: full report draft and figures; buffer for slipped runs	Report drafted
15 (Nov 30-Dec 1)	Final edits, code cleanup, reproducibility README	Submit Dec 1

Experiments end on Nov 20 so the last week and a half go to writing. If the core slips, E4 is cut first. The stretch goal starts only if E3-E5 finish early. E0-E2 are never cut.

4.3 Risks

vLLM’s connector interface changes between releases and may make it harder than expected to inject assembled KV; we pin one version and follow LMCache’s connector, and the core does not depend on vLLM. Reuse may lose more quality than the papers report: AgentKVShift’s own LoCoMo table puts CacheBlend at 0.290 F1 against 0.509 for full recompute (Mistral-7B, LiCoMemory, 10% budget), and if we see the same we report it. Finally, AgentKVShift runs on memory systems (A-Mem, LiCoMemory) with generated metadata while our memories are plain conversation chunks, so we report both absolute and relative F1 gaps.

5. Evaluation and testing method

Setup. Mistral-7B-Instruct-v0.3 on one PACE GPU, greedy decoding, the same saved retrievals for every configuration, one warm-up run and the median of three repetitions. Quality is LoCoMo F1, overall and per question category, with full prefill of the same prompt as the reference. Systems runs use the Zipf workload. Every result is stored with its configuration (vLLM version, GPU, k , budget, seed).

ID	Question	Stack	Measurement
E0	Is the implementation correct?	Transformers	Chunk re-rotated to its original position reproduces full-prefill logits; first-layer KV matches at any position
E1	How much quality does each method keep?	Transformers	LoCoMo F1 vs. recompute budget (0-30%), one line per method
E2	How much prefill is removed?	Transformers	Prefill time and tokens computed vs. k
E3	Does the connector cut latency?	vLLM	TTFT vs. k : full prefill, prefix caching, connector
E4	When does reuse beat prefix caching?	vLLM, Zipf	Throughput and TTFT as reuse rate goes from ~0% to ~100%
E5	Does the connector match the reference?	vLLM vs. HF	Method 1 F1 identical in both stacks

Validation. E0 is a hard gate: if re-rotation cannot reproduce full prefill where it should, every later number is

meaningless, so nothing else is reported until it passes. Unit tests cover RoPE round trips, hash collisions in the chunk store and cache assembly for out-of-order chunks. E5 ties the vLLM path to the Transformers path, and we compare our full-prefill and method 4 F1 with the numbers in [5] and explain any gap. Recomputed tokens are counted the same way for every method, including tokens touched at a selection layer. If time permits we ablate K-only versus K-and-V offsets in method 4, random versus top-divergence probes, and per-category F1, since both papers suggest multi-hop questions suffer most.

Success criteria. (1) CacheBlend and AgentKVShift stay within 2 F1 of full prefill at a 10-15% recompute budget, while re-positioning only scores clearly lower. (2) Those methods cut prefill time by at least $2\times$ when 6 or more entries are retrieved. (3) The connector beats both full prefill and prefix caching on TTFT when entries repeat in different orders. Missing a criterion is still a result, and the report will state what we measured.

6. Bibliography

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP*, 2023. arXiv:2309.06180. <https://github.com/vllm-project/vllm>
- [2] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, Y. Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864, 2021.
- [3] J. Hu, W. Huang, W. Wang, H. Wang, T. Hu, Q. Zhang, H. Feng, X. Chen, Y. Shan, T. Xie. EPIC: Efficient Position-Independent Caching for Serving Large Language Models. *ICML*, 2025. arXiv:2410.15332.
- [4] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, J. Jiang. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. *EuroSys*, 2025. arXiv:2405.16444.
- [5] N. P. Pandey, J. Kong, L. Hu, Q. Zhao, Y. Zhao, O. Gungor, H. Zhang, T. Rosing. AgentKVShift: Efficient KV Cache Reuse for Agentic Memory Systems. arXiv:2607.21604, 2026.
- [6] LMCACHE. <https://github.com/LMCACHE/LMCACHE> (issues #4476 and #5101 on the blend path).
- [7] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, Y. Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. *ACL*, 2024. arXiv:2402.17753. <https://github.com/snap-research/locomo>
- [8] L. Zheng et al. SGLang: Efficient Execution of Structured Language Model Programs. *NeurIPS*, 2024. arXiv:2312.07104.
- [9] I. Gim, G. Chen, S. Lee, N. Sarda, A. Khandelwal, L. Zhong. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. *MLSys*, 2024. arXiv:2311.04934.
- [10] Y. Liu et al. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. *SIGCOMM*, 2024. arXiv:2310.07240.
- [11] G. Xiao, Y. Tian, B. Chen, S. Han, M. Lewis. Efficient Streaming Language Models with Attention Sinks. *ICLR*, 2024. arXiv:2309.17453.
- [12] C. Packer et al. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.
- [13] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, Y. Zhang. A-Mem: Agentic Memory for LLM Agents. *NeurIPS*, 2025. arXiv:2502.12110.
- [14] H. Ye et al. KVCOMM: Online Cross-context KV-cache Communication for Efficient LLM-based Multi-agent Systems. *NeurIPS*, 2025. arXiv:2510.12872. <https://github.com/FastMAS/KVCOMM>
- [15] H. Jeon, H. Ha, S. Lee, B. Kang, J.-J. Kim. KVCMA: Efficient KV Cache Correction for Shared Context in Multi-Agent Systems. arXiv:2609.34060, 2026.
- [16] A. Q. Jiang et al. Mistral 7B. arXiv:2310.06825, 2023.